ActInf GuestStream 051.1 ~ Tommaso Salvatori "Causal Inference via Pred

GuestStream | Tommaso Salvatori | 1:27:29

Hello and welcome.
It's ActiveInference Guest Stream number 51.1 on July 28th, 2023.
We are here with Tomaso Salvatore and we will be having a presentation and a discussion
on the recent work, Causal Inference via Predictive Coding.
So thanks so much for joining.
For those who are watching live, feel free to write questions in the live chat and off
to you.
Thank you.
Thank you very much, Daniel, for inviting me.
I've always been a big fan of the channel and I've been watching a lot of videos, so I'm
quite excited to be here and be the one speaking this time.
So I'm going to talk about this recent preprint that I put out, which has been the work of
the last couple of months.
And it's a collaboration with Luca Vincetti, Amin Makarak, Beren Millic and Thomas Lukasiewicz.
And it's basically a joint work between Versus, which is the company I work for, the University
of Oxford and TUVN.
So during this talk, I will, this is basically the outline of the talk.
I will start talking about what predictive coding is and give an introduction of what it
is, a brief historical introduction, why I think it's important to study predictive coding,
even, for example, from the machine learning perspective.
I will then provide a small intro to what causal inference is.
And once we have all those informations together, I will then discuss why I wrote this paper, what
was basically the research question that inspired me and the other collaborators, and present the
main results, which are how to perform inference, so intervention and counterfactual inference, and how
to learn the causal structures from a given data set using predictive coding.
And then I will, of course, conclude with some, with a small summary and some discussion on why I
believe this work can be in fact impactful and some future directions.
So what is predictive coding?
Predictive coding is in general famous for being a neuroscience inspired learning method.
So a theory of how information processing in the brain works.
And very formally speaking, the theory of predictive coding can be described as basically having a
hierarchical structure of neurons in the brain.
And you have two different families of neurons in the brain.
The first family is the one in charge of sending prediction information.
So neurons in a specific level of the hierarchy send information and predict the activity of the

level below.
And the second family of neurons is that of error neurons.
And the error neurons, they send prediction error information up the hierarchy.
So one level predicts the activity of the level below.
This activity has some, this prediction has some mismatch, which with what actually going on in the level below.
And the information about the prediction error gets sent up the hierarchy.
So, however, predictive coding is, was actually not burned as a neuroscience, as a theory from, from the neurosciences, but it was actually initially developed as a method for signal processing and compression back in the fifties.
So the, the work of Oliver, Elias, which are actually contemporary of, uh, Claude Shannon, uh, of Shannon.
They realized that once we have a predictor, a model that works kind of, that is well in predicting data, sending messages about the error in those predictions is actually much cheaper than sending, uh, the entire message every time.
So, and this is how predictive coding was born.
So as a, uh, as a signal processing and compression mechanism in information theory back in the fifties, it was actually in the eighties, uh, that, that he became that exactly the same model was used in, uh, in neuroscience.
So with the work from Mumford or other works that, for example, explain how the, how the retina process information.
So we get prediction signals from the outside world, and we need to, to compress this representation and, uh, and have this internal representation in our neurons.
And the method is, uh, very similar, if not equivalent to the one that was used, the, that was developed by, by Elias and Oliver in the fifties.
Maybe what's the biggest paradigm shift happened in 1999, thanks to the work of Rao and Ballard, in which they, they are, uh, they introduced this concept that I mentioned earlier about hierarchical structures in the brain, where, uh, prediction information is top down and the error information is bottom up.
And something that they did that wasn't done before is that they, they explain and develop this theory about not only inference, but only, but also about how learning works in the brain.
So it's also a theory of how our synapses get updated.
And the last big breakthrough that I'm going to talk about in this brief historical introduction is from 1003, but is, uh, then he kept going in the, in the years after, uh, thanks to Carl Friston, in which basically he, he, he took the theory of Rao and Ballard and he developed, uh, he extended it and generalized it to, to the theory of generative models.
So basically the main claim that, that Carl Friston did is that predictive coding is an evidence maximization scheme of a, of a specific kind of generative model, which I'm going to, to, to introduce later as well.

So to make a, a brief summary in the, the first two, uh, kinds of predictive coding that I described,

so signal processing and compression and, uh, information processing in the retina and in the brain in general,

they are inference methods.

And the biggest, uh, change, the biggest, uh, revolution that we had in 1999.

So let's say in the 21st century is that predictive coding was seen as a learning algorithm.

So we can first compress information and then update all the synapses or all the latent variables that we have in our generative model to, to improve our generative model itself.

So let's, let's give some, uh, definitions that are a little bit more formal.

So predictive coding can be seen as a hierarchical Gaussian generative model.

So here is a very simple figure in which we have this hierarchical structure, which can be as deep as we want.

So this is a, uh, and signal, uh, prediction signals goes from, go from one latent variable XN to the following one.

And it gets transformed every time via function GN or GI.

And this is a generative model, as I said, and what's the marginal probability of this generative model?

Well, it's simply the probability of the last, uh, can you see my, my cursor?

Yes.

Right.

Yes.

Perfect.

So it's the generative model of the last vertex is the, sorry, probability distribution of the last vertex times the probability distribution of every other vertex conditioned on the activity of the vertex before or the, or the latent variable before.

I, I, I earlier said that is a Gaussian generative model, which means that those probabilities, they are in Gaussian form.

And every, and those function, function G in general, and, uh, especially since, for example, in a Ramballar paper and in all the papers that came afterwards, also because of the deep learning revolution, those functions are simply linear maps or, uh,

non-linear maps with activation functions or, uh, non-linear maps with activation function and an additive bias.

So we can, uh, we can give, uh, an informal definition of predictive coding and we can say that predictive coding is an inversion scheme for such a generative model where its model evidence is maximized by minimizing a quantity that is called a variation of free energy.

In general, uh, in general, uh, in general, uh, the goal of every generative model is to maximize model evidence, but this quantity is always intractable.

And, uh, and we have some, uh, techniques that allow us to, to approximate the solution.

And the one that we, that we use in predictive coding is that of minimizing aberration of free energy, which is a, uh, which is a lower bound of the model evidence.

In this work and actually in, uh, in, uh, in a lot of other ones.

So it's the standard way of doing it.

This minimization is performed via gradient descent.

Um, and, uh, yes, perform via gradient descent.

And there are actually other methods such as expectation maximization, which is often equivalent, or you can use some other message passing algorithms, such as belief propagation, for example.

And, uh, going a little bit back in time.

So we're getting a little bit about the statistical generative models.

We can see predictive coding, uh, as a, I mean, I said already a couple of times as a hierarchical model with, uh, neural activities.

So with neurons, latent variables that represent neural activities, the sender signal down the hierarchy and with error nodes or error neurons, the sender signal up the hierarchy.

So they send the error information back.

What's the variation of free energy of this class operated coding models?

It's simply the sum of the mean square error of all the error neurons.

So is the, is the sum of the, of the error of the total error squared.

And, uh, this representation is going to be useful in, uh, in the later slides and in how I'm going to explain how to use predictive coding to model causal inference, for example.

Why do you think predictive coding is important and is not, is a nice algorithm to study?

Well, first of all, as I said earlier, it optimizes the correct objective, which is the model evidence or marginal likelihood.

And, uh, then it does so by optimizing a lower bound, which is called the rational free energy, as I said.

And the racial free energy is interesting because it can be written as a sum of two different terms.

Which are an, each of those term, uh, optimizing it as, uh, as important impacts, for example, in, uh, in machine learning tasks or in general in learning tasks.

So one of those term forces memorization.

So in the second term, it basically tells, uh, forces the model to fit a specific dataset.

And the first term forces the model to minimize the complexity.

And as we know, for example, for the, from the, of, of, of, comes razor theory,

if we have two different models that perform similarly on a specific training set,

the one that we have to get and the one that is expected to generalize, uh, the most is the less complex one.

So updating, uh, generative model via rational free energy allows us to basically, uh, converge to the optimal, uh,

Ockham razor, uh, model, which is, which both memorizes a dataset, but it's also able to generalize very well on unseen, unseen data points.

So, uh, the second reason of why predictive coding is important is that it's actually not, uh,

uh, uh, doesn't have to be defined on hierarchical structure,

but it can be modeled on more complex and flexible architectures, such as directed graphical model with any shape or generalized even more to networks with a lot of cycles that resemble brain region.

And the end reason, the underlying reason is that you're not learning and predicting with a forward pass

and then back propagating the error, but you're minimizing an energy function.

And this allows basically every kind of hierarchy to be, uh, allows to go behind hierarchies and allow to, to learn cycles. And this is actually quite important because the brain is full of cycles. As we have some, uh, some information from some recent papers, uh, that are managed to map completely the, the brain of some animals such as fruit fly, the brain is full of cycles. So it makes sense to, to train our machine learning models or our, our models in general with an algorithm that allows us to train using cyclic structures. The third reason why predictive coding is interesting is that it has been formally proven that it is more robust than standard neural networks trained with back propagation. So if you have a neural network and you want to perform classification tasks, you, uh, predictive coding is more robust. And this is, uh, uh, interesting in tasks such as online learning, training on small datasets or continuous learning tasks. And the theory basically comes from the fact that predictive coding has been proved to approximate implicit gradient descent, which is a different version of the explicit gradient descent, which is the standard gradient descent used in the, in every single model basically. And it's a variation that is more robust.

I think, okay, I did a quite a long intro to predictive coding. I think I'm now moving to the second topic, which is causal inference. And, um, what's causal inference? Causal inference is a theory, theory is a very general theory that has been formalized the most by Judea Pearl. He's definitely the the most important person in the field of causal inference. He wrote some very nice books. For example, the book of Y is highly recommended if you want to learn more about this topic. And, uh, it basically tackles the following problem. So let's assume we have a joint probability distribution, which is associated with a Bayesian network. This is going to be a little bit the running example through all the paper, especially when you're with Bayesian networks of this shape. Those Bayesian networks, the, the variables inside, they can represent different quantities. So for example, a Bayesian network with this shape can represent the, the quantities on the right. So social economical statue, statue of an individual, its education level, its intelligence, and its income level. So, uh, something that classical statistics is very good at, and is, it's, uh, while, uh, most use application is to model observations or correlations. A correlation basically answers the question, what is D if we observe another, uh, variable C? So for example, in this case, what is, what's the income level, the expected income level of an individual, if I observe his education level? And of course, if that person has a, has a higher degree of education, for example, a master or a PhD, I'm expecting in general that person to have a higher income level. And this is a correlation. However, sometimes there are things that are very hard to observe, but they play a huge role in determining those quantities. So for example, it could be that the income level is much, much more, uh, defined by the intelligence of a specific person. And, uh, and maybe that, um, the intelligence, so if a person is intelligent, he's also most likely to have a higher education level. But still the, the real reason why the per, the income is high is because of the, of the IQ.

And this can be, this cannot be studied by simply correlations and has to be studied by,
by a more advanced technique, which is called an intervention.
An intervention basically answers the question is what is D if we change C to a specific value?
So for example, we can, we can take a, an individual and check his income level and then change its education level.
So intervene on this world and change his education level without touching his intelligence and see how much
is, uh, its income, uh, changes.
For example, if the income changes a lot, it means that the, the intelligence doesn't play a big role in, in this, the intelligence, but the education level does. If the income level doesn't change much, it means that maybe there's a hidden variable in this case, the intelligence that determines the income level of a person.
The third quantity important in causal inference is that of counterfactuals.
So for example, a counterfactual answers the question, what would D be had we changed C to a different value in the past?
So for example, we can see that the difference between interventions and counterfactuals
is that interventions act in the future. So I'm, I'm intervening in the world now to observe a change
in the future. While counterfactuals allow us to go back in time and change a variable back in time
and see how that change would have influenced the world we live in now.
And those are, uh, defined by Judea Pearl as the three levels of causal inference.
Correlation is the first level, intervention is the second level, and counterfactual is the third level.
So what are interventions? I'm going to define them more formally now,
now that I gave an intuitive definition. And I'm, I'm using this notation here, which is the same
actually toward all the presentation. So X is always going to be a latent variable.
S i is always going to be a data point or an observation. And V i is always going to be a vertex.
So every time you see V i, we're all, we are only interested in the structure of the graph, for example.
So let's assume we have a Bayesian model, which has the same structure of the, as the,
as the Bayesian model we, we saw in the previous slide.
Given that X3 is equal to S3, this is the observation we make,
statistics allows us to compute the probability or the expectation of X4,
which is the latent variable related to this vertex, given that X3 is equal to S3.
To perform an intervention, we need a new kind of notation, which is called the do operation.
So in this case, X4, we want to compute the probability of X4, given the fact that we
intervene in the word and change X3 to S3. And how do we do this? To perform an intervention,
Judea Pearl tells us that we have to, to have an intermediate step before computing a correlation,
is that first we have to remove all, to remove all the incoming edges to V3.
So we have to study not this Bayesian network, but this second one.
And then at this point, we are allowed to compute a correlation, as we, as we normally do.
And this is an intervention.

A counterfactual is a generalization of this, that as I said, lived in the past.

And they're computing using structural causal models.

So let's look at the results. A structural causal model is a tuple, which is conceptually similar to a Bayesian network.

But basically, we have this new class of variables on top, which are the unobservable variables they use.

So we have the Bayesian network that we had before, X1, X2, X3, X4.

But we also have those unobservable variables that depend on the environment.

You cannot control them. You can infer them, but they are there.

And f is a set of functions that depends on all the...

Basically, f of X3 depends on X1, because you have an arrow, on X2, because you have an arrow,

and on the unobservable variable that also influences X3.

So, yes, intuitively, you can see a...

You can think of a structural causal model as a Bayesian network with those unobservable variables on top.

And each unobservable variable only influences its own latent variable X.

So, for example, IU will never touch X1 as well.

U3 will only touch U3.

U1 will only influence X1 and so forth and so on.

So, performing counterfactual inference answers the following question.

So, what would X4 be at X3 being equal to another variable in a past situation, U?

And computing this counterfactual requires three different steps.

So, abduction is the...

is the computation of all the background variables.

So, in this step, we want to go back in time and understand how the environment,

the unobservable environment, was in that specific moment in time.

And we do this by fixing all the latent variables X to some specific data that we already have.

And performing this inference on the use.

Then we're going to use the U to keep the U that we have learned and perform an intervention.

So, a counterfactual can also be seen as an intervention back in time,

in which we know the environment variables U1, U2, and U4 in that specific moment.

And what's the missing step?

So, what would X4 be at X3 being equal to another data point in that specific situation?

Now we can compute a correlation.

And the correlation, we do it on the graph in which we have already performed an intervention,

using the environment variables that we have learned in the abduction step.

And this is a counterfactual inference.

This is the last slide of the causal inference introduction.

And it's about structural learning.

Basically, everything I've said so far relies on the fact that we know the causal dependencies among

the data points.
So, we know the structure of the graph.
We know which variable influences which one.
We know the arrows in general.
But in practice, this is actually not always possible.
So, we don't have access to the causal graph most of the times.
And actually, learning the best causal graph from data is still an open problem.
We are improving in this.
We are getting better.
But how to perform this task exactly is still an open problem.
So, as I said, basically, the goal is to infer causal relationships from observational data.
Given a dataset, we want to infer the directed acyclic graph that describes the connectivity
between the system and the variables of the dataset.
So, for example, here we have an example that I guess we are all familiar with because of the pandemic.
So, we have those four variables, age, vaccine, hospitalization, and city.
And we want to infer the causal dependencies among those variables.
So, for example, we want to learn directly from data that the probability of a person being hospitalized
depends on its age and on the fact whether it's vaccinated or not, and so forth and so on.
So, this is the end of the long introduction.
But I hope it was clear enough and I hope that I gave the basics to understand the results of the paper.
And now we can go to the research questions.
So, the research questions are the following.
First, I want to see whether predictive coding can be used to perform causal inference.
So, the predictive coding so far has only been used to perform to compute correlations in Bayesian networks.
And the big question is, can we go beyond correlation and model intervention and counterfactual
in a biological, plausible way?
So, in a way that it's, for example, simple, intuitive, and allow us to only play with the neurons
and not touch, for example, the huge structure of the graph.
And more in practice, more specifically, the question becomes, can we define an operative coding-based
structural causal model to perform interventions and counterfactuals?
The second question is, as I said, that having a structural causal model assumes that we know the
structure of the Bayesian network.
So, it assumes that we have the errors.
Can we go beyond this and use predictive coding networks to learn the causal structure of the graph?
Basically, giving positive answers to both those questions would allow us to use predictive coding
as an end-to-end causal inference method, which basically takes a dataset and allow us to test interventions
and counterfactual predictions directly from this dataset.
So, let's tackle the first problem.

So, causal inference via predictive coding, which is also the section that gives the title to the paper, basically.
So, here I will show how to perform correlations with predictive coding, which is already known,
and how to perform interventional queries, which I think is the real question of the paper.
So, here is a causal graph, which is the usual graph that we had.
And here is the corresponding predictive coding model.
So, the axes are the latent variables and correspond to the neurons in a new
neural network model.
And the black arrow passing from prediction information from one neuron to the one down
the hierarchy.
And every vertex also has this error neuron, which passes information up the hierarchy.
So, the information of every error goes to the value node up the hierarchy and basically tells
it to correct itself to change the prediction.
So, to perform a correlation using predictive coding, what you have to do is that you take an observation
and you simply fix the value of a specific neuron.
So, if you want to compute the probability of X4 given X3 equal to S3,
we simply have to take X3 and fix it to S3 in a way that it doesn't change anymore
and run an energy minimization.
And this model, by minimizing, by updating the axes via a minimization of the variational free energy,
allows the model to converge to a solution to this question.
So, the probability or the expected value of X4 given X3 equals S3.
But how do I perform an intervention now without acting on the structure of the graph?
Well, this is basically the first idea of the paper.
This is still how to perform a correlation.
So, fix S3 equal to X3 is the first step in the algorithm.
And the second one is to update the axes by minimizing the variational free energy.
An intervention which, in theory, corresponds in removing those arrows
and answers to the question the probability of X4
by performing an intervention, so do X3 equal S3.
Imperative coding can be performed as follows.
So, I'm going to write the algorithm here.
So, first, as in a correlation, you fix S3 equal to the observation that you get.
Then, this is the important step.
You have to intervene not on the graph anymore, but on the prediction error,
and fix it equal to zero.
Having a prediction error equal to zero basically
makes sense, meaning the meaningless information up the hierarchy,
or actually sends no information up the hierarchy, because it basically tells you that
the prediction is always correct.

And the third step is to, as we did before, to update the axes,
the unconstrained axes, so X1, X2, X4, by minimizing the variational free energy.
As I will show now experimentally, by simply doing this little trick of setting a prediction error
that the theory to be equal to zero, prevents us to actually act on the structure of the graph,
as the theory of Ducalculus does, and to infer the missing
the variables after an intervention, by simply performing aberration of free energy minimization.
So, what about counterfactual inference?
Counterfactual inference is actually easy once we have defined how to do an intervention.
And this is because, as we saw earlier, performing a counterfactual is similar to performing an
intervention in a past situation after you have inferred the unobservable variables.
So, as you can see in the plot I showed earlier about the abduction, action, and prediction steps,
the action and prediction steps, they did not have those two ROOs.
They were removed.
Criticoding allows us to keep the ROOs in the graph, and perform counterfactuals by simply performing an adduction step, as it was done earlier, an action step in which we simply perform an intervention on the single node.
So, we fix the value node and we set the error to zero.
And run the energy minimization, so minimizing the ratio of free energy to compute the prediction.
So, I think this is like an easy and elegant method to perform interventions and counterfactuals.
And, yeah, so I think the thing we have to show now is whether it works in practice or not.
And we have a couple of experiments.
And I'm going to show you now two different experiments.
The first one is merely a proof-of-concept experiment that shows that the predictive coding is able to perform intervention and counterfactuals.
And the second one actually shows a simple application in how interventional queries
can be used to improve the performance of classification tasks on a specific kind of
predictive coding networks, which is that of a fully connected model.
So, let's start from the first one.
So, how do we do this task?
So, given a structural causal model, we generate training data and we use it to learn the weights,
so to learn the functions of the structural causal models.
And then we generate test data for both interventional and counterfactual queries.
And we show whether we are able to converge to the correct test data using predictive coding.
And, for example, here, those two plots represent the interventional and counterfactual queries
of this specific graph, which is the butterfly bias graph, which is a graph that is often used in
in testing whether causal inference, whether interventional and counterfactual techniques work.
It's as simple as that.
But in the paper, you can find a lot of different graphs.

But in general, those two graphs, those two plots show that the method works, show that the mean absolute error between the interventional and counterfactual quantities we compute and the interventional and counterfactual quantities from the original graph are close to each other.
So the error is quite small.
The second experiment is basically an extension of an experiment I proposed in an earlier paper, which is the learning on arbitrary graph topologies that I wrote last year.
In that paper, I basically proposed this kind of network as a proof of concept, which is a fully connected network, which is in general the worst neural network you can have to perform for machine learning experiments.
Because given a fixed set of neurons, basically the
you have every pair of neuron is connected by two different synapses.
So it's the model with the highest complexity possible in general.
The good thing is that since you have a lot of cycles, the model is extremely flexible in the sense that you can train it, for example, on a minst image and on a data point and on its label.
But then the way you can query it, thanks to the information going back, is you can query in a lot of different ways.
So you can perform classification tasks in which you provide an image and you run the energy minimization and get the label.
But you can also, for example, perform generation tasks in which you give the label, run the energy minimization and get the image.
You can perform, for example, image completion in which you give half the image and let the model convert to the second half and so forth and so on.
So it's basically a model that learns the statistics of the data set in its entirety without being like focused on classification or generation in general.
So this flexibility is great.
The problem is that because of this, every single task doesn't work well.
So you can do a lot of different things, but none of them is done well.
So this is a problem.
And here I want to show how using interventional queries instead of standard correlational queries or conditional queries slightly improves the results of those classification tasks.
So what are the conjecture reasons of this test accuracy on those tasks not being so high?
The first two reasons are that the model is distracted in correcting every single error.
So basically you present an image and you would like to get a label, but the model is actually updating itself to also predict the error in the images.
And the second reason, which is the one I said, is that the structure is far too complex.
So again, from an Occam razor argumentation, this is the worst model you can have.
So every time you have a model that fits a data set, that model is going to be less complex

than this one that is going to be preferred.
But in general, just to study it, the idea is can querying this model via interventions be used to improve the performance of those fully connected models?
Well, the answer is yes.
So here is how I perform interventional queries.
So I present an image to the network.
So I create a network.
I fix the error of the pixels to be equal to zero.
So this error doesn't get propagated in the network.
And then I compute the label.
And as you can see, the accuracy improves, for example, from 89,
using the standard query method of predictive coding networks, to 92,
which is the accuracy after the intervention.
And the same happens for fashion means.
So I think that a very legit critique that probably everyone would think when seeing those plots is that, okay, you improve on means from 89 to 92, it still sucks, basically.
And yeah, it's true.
And actually, in the later slides, I'm going to show how to act on the structure of this
of this fully connected model will improve the results even more until the point they reach a performance that is not even close to state-of-the-art performance, of course,
but it's still up to a level that becomes basically acceptable and worth investigating.
So, yes, so this was the part about causal inference using predictive coding.
And I guess to summarize, I can say that the interesting part of the results I just showed is that I showed that predictive coding is able to perform interventions in a very easy and intuitive way because you don't have to act on the structure of the whole graph anymore.
Sometimes those functions are not available, so forth and so on, but you simply have to
you simply have to intervene on a single neuron, set its prediction error to zero,
and perform an energy minimization process.
And this extended allowed us to define predictive coding-based structural causal models.
Now we move to the second part of the work, which is about structural learning.
So, structural learning, as I said, deals with the problem of learning the causal structure of the model from observational data.
So, this is actually an old problem that has been around for decades and
and has always been, until a couple of years ago, tackled using combinatorial search methods.
The problem with those combinatorial search methods is that
their complexity grows double exponentially.
So, as soon as the data becomes multi-dimensional and the Bayesian graph that you want to learn grows in size,

learning it is incredibly slow.
The new solution that came out actually a couple of years ago in a New Reap's paper from 2018 showed that it's possible to actually learn this structure not using a combinatorial search method, but by using a gradient-based method.
And this was basically, this killed the problem in general, because now you can simply have a prior on the parameters, which is the prior they proposed that I'm going to to define a little bit better in this slide, around gradient descent.
And even if you have a model that is double, triple the size, the algorithm is still incredibly fast.
And for this reason, this paper is a, this is, yeah, I think it's kind of new, and I think already has around 600 citations or things like that.
And every paper that I'm seeing now about causal inference and learning causal structure of the graph uses their method. It just changes a little bit. They, they find faster or slightly better inference methods, but still they all use the prior, the, this paper defined. And I do as well, and we do as well.
So here we, we define a new quantity, which is the agency matrix. The agency matrix is simply a matrix that encodes the connections of the model. So it's a binary matrix and in general is a binary matrix.
Then of course, when you do gradient based optimization, you, you make it continuous. And then you have some threshold at some point that basically kills an edge or, or set it to one.
And the entry IJ is equal to, to one. If we have, if the Bayesian graph has an edge from, from vertex I to vertex J or zero otherwise. So for example, this agency matrix here represents the connectivity structure of this Bayesian network. And basically this method tackles two, two problems that we want about this, about learning the structure of the Bayesian network. The idea is that we start from a fully connected model, which conceptually is, is similar, actually is equivalent to the predictive coding network I defined earlier, which is fully connected. So you have, you have a lot of vertices and every pair of vertices is connected by, by two different edges. And you simply want to prune the ones that are not, not needed. So it can be seen as a, as a method that performs model reduction. You start from a big model and you want to make it small. So what's the first ingredient to, to reduce models? Well, is of course, sparsity. And what's the prior that everyone uses to, to make a model more sparse? Is the Laplace prior,
which in machine learning is simply known as the L1 norm, which is defined here. The solution that the, this paper that I mentioned earlier proposed is to add the second prior on top, which enforces what, what's probably the, the biggest, uh, characteristic of Bayesian networks, the, on which you want to perform causal inference is that you want them to be acyclic. And basically they show that acyclicity can be imposed on, uh, on an agency matrix as a prior. And it has this, the, this shape here. So it's the trace of the matrix that is the, uh, the exponential of A times A, where A is the agency matrix again.
And, uh, basically this quantity here is, is equal to zero if, and only if the Bayesian network or this, or the, whatever graph you're considering is acyclic.
So I'm going to use this in, uh, in some experiments. So those two, to force those two priors on the, uh, on different kinds of Bayesian networks, and I'm trying to merge them with the techniques we

proposed earlier about performing causal inference via predictive coding.
So I'm going to present two different experiments. So one is, uh, is a proof of concept, which is the standard experiments, uh, showed in all the structural learning tasks, which is the, uh, inference of the correct Bayesian network from data. And then I'm going to, to build on top of the classification experiments I showed earlier and, uh, and show how actually those priors allow me, allow us to improve the classification accuracy, the test accuracy of fully connected predictive coding models.
So let's move to the first experiment, which is to infer the structure of the graph and the experiments, they all follow basically the same pipeline in, uh, in all the papers in the field.
The first step is to generate, uh, a Bayesian network from random graph.
So basically normally the two random graphs that everyone tests are Erdos-Reni graphs and scale-free graph.
So you, you generate those big graphs that normally have 20, 40, 80, 80 different nodes and some edges that you, that you sample randomly. And you use this graph to generate a dataset.
So you sample, for example, uh, N big N data points.
And what you do is that you take the graph that you, the, they have generated earlier and you throw it away. You only keep the dataset. And the task you want to solve now is to, is to learn, is to have a training algorithm that basically allows you to, to retrieve the structure of the, of the graph you have thrown away.
So the way we do it here is that we train a fully connected predictive coding model on this dataset D, using both the sparse and the acyclic priors we have defined earlier.
And, and see whether actually the, the graph that we converge to after pruning away the, uh, the entries of the agency matrix that are smaller than a certain threshold is similar to that of the, of the initial graph.
And Erdos-Rensols show that this is actually the case.
So this is an example. And, uh, I show many different parametrization and dimensions and, and things like that in the, in the paper, but I think those two are the most representative examples with an Erdos-Rensols graph and a free scale graph, uh, with 20 nodes.
And here on the left, you can see the ground truth graph, which is the one sampled randomly. And on the right, you can see the graph, uh, the predictive coding model has learned from, from the dataset. And as you can see, they are quite similar. It's still not, it's still not perfect. So there are some, uh, there are some errors, but in general, the structure is, uh, they work quite well. We also have some quantitative experiments that, that we, that I don't show here, because they're just huge tables with a lot of numbers. And I thought it was maybe a little bit too much for the presentation, but, but there's also that they perform similarly to, to contemporary methods. Also, because I have to say, like most of the quality comes from the, from the acyclic prior that was introduced in, in 2018.
The second class of experiments is the, are classification experiments, which as I said, are the extensions of the, the one I showed earlier. And the idea is to use structured learning to

improve the classification on the, the classification results on the means and fashion means data set.
starting from a fully connected graph.
So what I did is that I divided the fully connected graph in clusters of neurons.
So one big cluster is the, is the one related to the input.
And all the small, then we have some specific number of hidden clusters.
And then we have the label cluster, which is the, the class, the cluster of neurons that are
supposed to give me the, the label predictions.
And I've trained them using for using the first time, the sparse prior only.
So the idea is what if I, if I prune the connections I don't need from a, from a model
and, and learn a sparser model, does this work?
Well, the answer is no, it doesn't work.
And the reason, and the reason why is that you, at the end, the graph that you converge with is
actually the generate.
So basically the model learns to predict the label based on the label itself.
So it discards all the information from the input and only keeps the label.
And as you can see here, the label Y predicts itself or in other experiments, when you change the
parameters, you have that Y predicts X0, that predicts X1, that predicts Y again.
So what's the, what's the solution to, to this problem?
Well, the solution to this problem is that we have to, to converge to an acyclic graph.
And so we have to add something that prevents acyclicity.
And what is that?
One is of course the one I already proposed.
And then I, I show a second technique.
So the first one uses the acyclic prior defined earlier.
And the second one is a, is a novel technique that actually makes use of negative examples.
So a negative, a negative example in this case is simply a data, a data point in which you have an
image, but the label is wrong.
So here, for example, we have an image of a seven, but the label that I
give in the model is a two.
And, and the idea is, is very simple in, it has, has been used in a lot of works already.
So every time the model sees a positive example, it has to increase, to, to minimize the relation
of free energy.
And every time it has a, it sees a negative example, it has to increase it.
So we want the error, this quantity to be minimized.
And actually with a lot of experiments and a lot of, uh, of experimentations, we saw that the two
techniques basically first lead to the, to the same results and second lead to the same graph as well.
So here are the, here are the new results on minst and fashion minst using the two techniques that I,
that I just proposed.

And, and now we move to some, which are still not, uh, great, but definitely more reasonable test accuracies.
So here we have a test error of 3.17 for minst and a test error of 13.98 for fashion minst.
And actually those can be, those results can be much improved by learning the structure of the graph on minst.
And then fixing the structure of the graph and, and do some form of fine tuning.
So if you fine tune the model on the correct hierarchical structure, at some point you reach the test accuracy, which is the one you would expect from a hierarchical model, but those ones are simply the one, the fully connect connected model as naturally converged to.
So, so for example, from a test error of 18.32 of the fully connected model train on fashion minst by simply performing, uh, correlations or conditional queries, which is the standard way of querying operative coding model. Adding interventions and the acyclic prior together makes this, uh, this test error much lower and we can observe it for, for minst as well.
I'm now going a little bit into details on, uh, on this last experiment and on how the acyclic prior acts on the structure of the graph.
So I perform, I perform an experiment on a, on a new dataset, which is, I mean, calling it a new dataset, it may be too much is that I called it a two means dataset in which you have the input point is formed of, uh, of two different images and the label only depends on the second image on the first image. Sorry. So the idea here is, is the structure of the model, the acyclicity, the acyclicity prior and things like that, able to recognize that the second half of the image is actually meaningless in, uh, in performing, in, in learning the, in performing classification.
How does training behave in general? Like for example, we have this, uh, input, uh, input node, output node, and only the nodes are fully connected. Can the model converge to a, uh, two hierarchical structure, which is the one that we know performs the best on, uh, on classification tasks? Well, here is, uh, is an example of a training method of a training run. So at c0, which is the beginning of training, we have this model here. So s0 corresponds to the, to the seven. So to the first image, s1 corresponds to the second image, again, we have the label y and all the latent variables x0, x1, x2, and the model is fully connected. So the agency matrix is, is full of ones. There are, there are no zeros. We have self loops and things like that. We train the, the model for a couple of epochs until, and what we note immediately is that for example, the, the model immediately understands that the four is not needed to perform classification. So it doesn't, uh, so every outgoing node from the, from the second input cluster is removed. And something that you understand is that this is, this cluster is the one related to the output. So we have a, we have a linear map from s0 to y directly, which is this part here. But we know that actually a linear map is not the best map for, for performing classification on minst. So we, we need some hierarchy. We need some depth to, to improve the results. And as you can see, this line here is the, is the accuracy, which up to this point, so up to c2 is similar to, um, so it's, it's 91%, which is slightly, slightly better than linear classification. But once you go on with the training, the model understands that it needs some hierarchy to better

fit the data. So you, you see that this arrow starts getting stronger and stronger over time until it, it understands that the linear map is not actually really needed and it removes it. And, uh, so, so the model you converge with is a model that starts from a zero, goes to a hidden node, and then goes to the, to the label with a very weak linear map, which actually gets removed. If you, if you set a threshold of, uh, if you set a threshold of, for example, 0.1, 0.2, at some point, the linear map gets forgotten and everything you end up with is with, uh, is with a hierarchical network that has, that, uh, so it has learned the correct structure to, to perform classification tasks, which is hierarchy. And it has also learned that the second image didn't play any role in defining the, the test accuracy. And this is all, this is all performed. Also, all those jobs are simply performed by, performed, uh, by one free energy minimization process. So you initialize the model, you define the free energy, you define the priors. So the, the sparse and the cyclic prior, you run the, the energy minimization and you converge to hierarchical, to a hierarchical model, which is well able to perform classification on means. And then if you then perform some fine tuning, you reach very competitive results as you do in feed forward networks with the, with backpropagation. But I think that's not the interesting bit. The interesting bit is that you like all this process, this process altogether of intervention and, uh, cyclicity allows you to take a fully connected network and converge to a hierarchical one that is, that is able to perform classification with good results.

And, um, yeah, it's, uh, that's basically it. I'm now, oh yeah. Wow. I've talked a lot. And I'm, uh, this is the conclusion of the talk, which is, uh, I'm basically doing a small summary. And I think the, the important takeaway, if I have to give you in one sentence of this paper, is that predictive coding is a belief updating method that is able to perform end-to-end causal learning. So it's able to perform interventions to learn a structure from data and, uh, and then perform interventions and counterfactuals. So causal inference in natural inefficiency model interventions by simply setting the prediction error to zero. So it's a, it's a very easy technique to perform interventions. And you simply only have to touch one neuron. You don't have to act on the structure of the graph. You can, you can use it to perform, to, to create structure causal models that are biologically plausible. It is able to learn the structure for, uh, from data, as I said, maybe a lot of times already. And, um, and a couple of sentences about future works is that something that would be nice to do is to improve the performance of the model we, we have defined, because I think it performs reasonably well on a lot of tasks. So it performs reasonably well on structural learning, on forming intervention and counterfactuals. But actually, if you look at state-of-the-art model, there's always like a very specific method that performs better in, in the single task. So it would be interesting to see if we can reach those levels of performance in, uh, in specific tasks by, by adding some tricks on or some, uh, or, or some new optimization methods and to generalize it to, to dynamical systems, which are actually much more interesting, the static systems. So such as dynamical causal models and, uh, or other techniques that allow you, that allow you to perform causal inference in systems that move. So an action taken in a specific time step influences another node in a later time, time, time step, which is, uh, basically Granger causality. Yeah. Uh, that's it. And, uh,

thank you very much.
Thank you. Awesome. And very comprehensive presentation. That was really, I think you're muted.
Sorry, muted on zoom, but yes, thanks for the awesome and very comprehensive presentation.
There was really a lot there. And there was also a lot of great questions in the live chat.
So maybe to warm into the questions, how did you come to study this topic? Were you studying causality and found predictive coding to be useful or vice versa? Or how did you come at this intersection?
I actually have to say that the first person that came out with this idea was, uh, was Barron.
So, so like, like, I think a year and a half ago, even more, he wrote like a page with this idea.
And then it got forgotten and no one picked it up. And, uh, and last summer I started getting curious about causality and, uh, um, I, I read, for example, the book of why I said, listening to podcasts,
I don't know the standard way in which you get interested in a topic.
And, uh, and I remember this, this idea from Baron and proposed it to him. And, uh, and I was like, why don't we expand it and, and, uh, and actually make it a paper. So I, I involved some people to happen with experiments. And, uh, and this is the final result at the end.
Awesome. Cool. Yeah. Um, a lot to say, I'm just gonna go to the live chat first and address a bunch of different questions. And if anybody else wants to add, I'm gonna turn the light on first. Cause I'm, I think I'm getting in the dark more and more. Yes. Who said active inference can't solve the dark room issue? Oh yes. Here we are. So would you say the light switch caused it to be lighter?
Yeah, I think so. No issues here. Um, okay. ML Don wrote, since in predictive coding, all distributions are usually Gaussian. The bottom up messages are precision weighted prediction errors, where precision is the inverse of the Gaussian covariance. What if non-Gaussian distributions are used?
Is, um, basically the general method stays. The different, the main difference is that you, you don't have prediction errors, which, uh, as was correctly pointed out is the, basically the derivative of the variational free energy. If you have Gaussian assumptions, yeah, you even have that single quantity to set to zero and you probably will have to act on the structure of the graph to perform interventions. And also you, uh, and colleagues had a paper in 2022 predictive coding beyond Gaussian distributions that, that looked at some of these issues, right?
Yes. Yes, exactly. So the paper was a little bit, the idea behind the paper is, uh,
and we model transformers. That's the biggest motivation using predictive coding. And the answer is, uh, is not because the, the attention mechanism has a softmax at the end and softmax calls to, uh, like not to Gaussian distribution, but to, yeah, to softmax distribution. The, I don't get the name now, but yes. And, uh, so yes, that's a generalization. It's a little bit tricky to call it. Once you remove the Gaston assumption is a little bit still tricky to call it predictive coding. So it's, uh, so for example, like talking to, uh, to Carl Freestone, either he, like, predictive coding is only if you, if you have only Gaussian Gaussian assumptions, but yes, that's more a philosophical debate than, uh, that. Interesting. And another, I think, topic that that's definitely of, of great interest is similarities and differences between the attention apparatus in transformers and the way that attention is described from a neurocognitive perspective and from a predictive processing precision weighting

angle. What do you, what do you think about that?
Well, the idea is that, um, yeah, I think about it is that in, from a predictive processing and, uh, and also operational inference perspective, attention can be seen as a, as a kind of structure learning problem. There's a, I think there's a recent paper from, uh, from Chris Buckley's group that shows that there should be, there should be a reprint on archive in which basically they show that the attention mechanism is simply learning the, the precision on the, on the weight parameters specific to a, to a data point. So this precision is not a, it's not a, it's not a parameter that is in the structure of the model. So it's not a model specific parameter, but it's a fast changing parameter like the value nodes that gets updated while minimizing the varational free energy. And once, once you've minimized it and computed, then you throw it away and from the next data point you have to recompute it from scratch. So yes, I think the, the analogy computation wise is, uh, the attention mechanism can be seen as a kind of structure learning, but a structure learning that is data point specific and not model specific. And I think if you want to generalize a little bit and go from, uh, from the attention mechanism in transformers to the attention mechanism in cognitive science, the, uh, I feel they're probably too different to like to draw similarities. And, uh, I think the structural learning analogy and the, how important one connection in is with respect to another one probably does job much better.
Cool. Great answer. Okay. ML Don asks in counterfactuals, what is the difference between hidden variables X and unobserved variables? You, the difference is that you can, uh, I think the main one is that you cannot observe the use. You can use them because you can, you can compute them and fix them, but you cannot, the idea is that you have no control over them. So the use, the use should be seen as a environment specific variables that they are there. They, they influence your process. Okay. Because the, for example, when you go back in time, the environment is different. So the idea is, for example, if you like going back to the, to the example before of the, of the expected income of a person with a specific intelligence of education, uh, uh, education degree, the idea is that if I want to, to see how much I will learn today with, uh, with, uh, with, uh, I don't know, with a master degree is different with respect to how much I would earn 20 years ago with a master degree is different. For example, here in Italy, with respect to other countries and all those variables that are not under your control, you cannot model them using your Bayesian network, but they are there. Okay. So you, you cannot ignore them when you, when you want to draw conclusions. So it's, yeah, it's basically everything that you cannot control. You can, you can infer them so you can, you can perform a counter, counterfactual inference back in time and say, Oh, 20 years ago, I would have earned this much if I, if I was this intelligent at this degree on average, of course, and, but it's not that I can change the government policies towards jobs or the, or things like that. It's a deeper counterfactual. Yes, exactly. So yeah, those are the use. Awesome. All right. Have you implemented generalized coordinates in predictive coding?
No, no, I've, no, I've never done it. I've, uh, I've studied it, but I've, uh, I've never implemented it. I know they tend to be unstable and, uh, and it's very hard to make them stable. I think that's the,

that's the takeaway that I got from talking to people that have implemented them.
But, but yeah, yeah. I'm aware of some papers that came out actually recently about, about them that, that tested on some racial out and coder style. Actually, I think still from Baron. There's a, there's a paper out there that came out last summer, but, but no, I've never played them with them myself.
Cool. From Bert, does adding more levels in the hierarchy reduce the distraction problem of predicting input. Adding more level in, uh, in which sense, because the distraction problem is given by cycles. So basically you, you provide an image and the fact that you have, uh, so edges going out of the image, going in the, in the neurons, and then other edges going back. The, this basically creates the fact that you have a, that the error of that those, basically these ingoing edges to the pixels of the image, they create some prediction errors. So you have some prediction errors that get spread inside the model. And that's yeah. And this problem I think is general of cycles and it's probably not related to hierarchy in general. So it is, it is the two incoming edges to the pixels. If you don't have incoming edges, you have no, uh, no distraction problem anymore. Cool. And, and the specification of the acyclic network through the trace operator, that's a very interesting technique. And when was that brought into play? Uh, as far as I know, I think it came out with the paper I, I cited in 2018. I, I don't know, at least in the causal inference literature, I'm, I'm not aware of any previous methods. I would say no, because then, I mean, that's the highly cited paper. So I would say they came out with that idea. Wow. Yeah. That's, that's quite nice that you can do gradient descent and learn the structure. I think that's a, that's a very powerful technique. Yeah. Sometimes it's like, when you look at when different, um, features of Bayesian inference and causal inference became available, it's really remarkable. Like why, why, why hasn't this been done under a Bayesian causal modeling framework? It's like,
because there's only been like five to 25 years of this happening. And so that's very, very short.
And also it's relatively technical. So there's relatively few research groups engaging in it.
And, um, it's just really cool what it's enabling. No, yes, yes, exactly. I mean,
that's also, I think the exciting part of this field a little bit that is, uh, I mean, there are definitely breakthroughs out there that, that still have to be discovered and probably like,
cause for example, like for as much as a breakthrough that paper was, they found, uh,
uh, like they simply found out the right prior for a cyclic structures. Okay. It's a, yeah. I mean,
I, I, I don't know exactly, but it may be an idea that you have in, in one afternoon. I don't know about the story of the, how the authors came up with that, but could potentially be that they, they, they're there, they're on the whiteboard. You're like, oh, that actually works. That's a huge breakthrough. And I simply, uh, define the prior. And also a lot of these breakthroughs,
they, they don't just stack. It's not like a, uh, uh, a tower of blocks. They, they, they layer and they compose. So then something will be generalized to, um, generalized coordinates or generalized synchrony or arbitrarily large graphs or, um, sensor fusion with multimodal inputs. And it's like, those all blend in really satisfying and effective ways. So, so even little things that,
again, someone can just come up with in a moment, um, can really have impact. Um, okay. ML Don says,

thanks a lot for asking my questions and thanks a million to Tomaso for the inspiring presentation.
So nice. Thank you very much. And then Bert asks, how would language models using predictive coding differ from those using transformers? Um, okay. I think that actually,
if I would have to build today a language model using predictive coding, I would still use transformers.
So the idea is that for example, if you have a, let's say this, uh, hierarchical, uh, graphical model
or these, or these hierarchical, um, Bayesian network, I've defined in the, in, in the very first
slide, one arrow to encode a function, which is the linear map. Okay. So one arrow was simply the
multiplication of a, of the vector encoded in the latent variables times the, this weight matrix that
you can then make nonlinear and things like that, but that can be actually something much more complex.
The, the function encoded in the arrow can be a convolution, can be an attention mechanism.
So, so actually how I would do it, I will still use the, I mean, which is actually the way we did it in, uh,
in, in the Oxford group last year is that we, we had exactly the structure. Every arrow is a transformer now.
So one is the attention mechanism and the, the next one is the feed forward network as transformers.
And basically the only difference that you have is that those variables, you want to compute the
posterior and you make those posteriors independence independent via, via mean field approximation.
So basically you follow all the steps that allow you to, to converge to the barrier, to the
variation of free energy of predictive coding. But the, the way, the way you compute predictions
and the way you, you send signals back is, uh, is done via transformer.
So I will still use transformers in general. I mean, they work so well that I, I don't think that
we can be arrogant and say, oh no, I'm going to do it better via a purely predictive coding way.
Structure learning is a way to do it, but it will still approximate transformers anyway.
I'm sorry, you said structure learning would approximate the transformer approach?
Yes. The structure learning I mentioned earlier in, uh, when, when someone, uh,
has the similarities between predictive coding and the attention mechanism.
Very, yeah. Very interesting. Um, one thing I am wondering from ML, I could not see the concept
of depth in the predictive coding networks you mentioned most likely I missed it. The
definition provided for predictive coding involved the concept of depth.
What did you mean by depth?
No, yes, it's true. It's, uh, cause the, the standard definition, uh, as I said, multiple times is, uh,
is hierarchical. You have predictions going one direction and prediction error going the opposite
direction. Basically what, uh, what we did in, in this paper and also in the last one in, uh, which
is called the learning on arbitrary graph topologies via creative coding is that we can consider depth,
uh, like as a, as independent, uh, basically pair of latent variable latent variable and arrow.
And you have predictions going that direction and prediction error going the other, but then you can
compose these in how many, uh, a lot of ways. So you can, uh, you can, so basically this composition
doesn't have to be hierarchical in the end. Can have cycles. So then you, you can, for example, plug in another
uh, another latent variable to the first one and then connect the other two and you can have a

structure that is, that is as entangled as you want. So for example, in the, in the other paper, we trained, uh, uh, a network that has the shape of a brain structure. So we have a lot of brain regions that are sparsely connected inside and sparsely connected among each other. And, and there's, there's nothing hierarchical there at the end, but you can still train it by minimizing a relation of free energy and by minimizing the, the total prediction error of the network. Hmm. So you could have for a given motif in a entangled graph, you might see three successive layers that when you looked at them alone, you'd say, oh, that's a three story building. That's a three layer model that has a depth of three. But then when you take a bigger picture, there isn't like an explicit top or an explicit bottom to that network.

Yes, exactly. And this is basically given by the, by the fact that every operation in predictive coding networks is, uh, is strictly local. So, so basically every message passing, every prediction and every prediction error that you send, you only send it to the very nearby neurons.

Okay. And whether the global structure is actually hierarchical or not, the, the, the single message passing doesn't even see that.

David Pérez

I guess that's sort of the hope for learning new model architectures is the space of what is designed top down is very small. And a lot of models in use today, albeit super effective models. Um, although you could ask effective per unit of compute or not, that's a second level question, but a lot of effective models today do not have some of these properties of predictive coding networks, like their capacity, um, to, to use only local computations, which gives biological realism, um, or, uh, just spatio temporal realism, but also, um, may provide a lot of advantages in like federated compute or distributed computing settings.

No, yes, exactly. I completely agree. Because, uh, I think the idea in general is that and, uh, I don't know if that's going to be an advantage. I think it's very promising exactly for the reasons you said. And, uh, and the reason is that today's models trained with backpropagation, you can basically, uh, summarize them as a, as a model trained backpropagation is a, is a function because basically you have a map from input to output and backpropagation basically spreads information back from its computational graph. So every neural network model used today is a function. While predictive coding and, uh, and not only predictive coding, like the old class of functions that, uh, class of methods that train in, uh, using local computations and actually work by minimizing a global energy function, they, they're not limited to model functions from input to output. They actually model something that, that kind of resembles physical systems.

So you have a physical system, you fix some values to, uh, to whatever input you have and you let the system converge and then you read some other value of, uh, neurons or variables that are supposed to be output. But this physical system doesn't have to be a feed forward map, doesn't have to be a function that has a, an input space and an output space and that's it. So the class of models that you can learn is, uh, so basically you can see like, uh, feed forward models and functions and then a much bigger class, which is that of physical systems. Whether there's something interesting out here,

I don't know yet because the functions are, are working extremely well. We are seeing those days with backpropagation is, uh, they work crazy well, but so, yeah, I don't know if there's anything interest, interesting in the big part, but the big part is, is quite big. Okay. There are, there are a lot of models that you cannot train with backpropagation and you can train with predictive coding or background propagation or other methods. That is super interesting. Certainly biological systems, physical systems solve all kinds of interesting problems. Um, but there's still no free lunch and ant species that does really well in this environment might not do very well in another environment. And so, um, out there in the, uh, in the hinterlands, there might be some really unique, special algorithms that are not well described by being a function yet still, uh, provide like a procedural way to, to, uh, implement heuristics, which might be extremely, extremely effective. No, yes, yes, exactly. And, uh, yeah. And I think this has been most of my focus of research during my PhD, for example, like finding this application that is like out here and not inside the, uh, the functions. Cool. Well, where does this work go from here? Like what directions are you excited about and how, how do you see people in the active inference ecosystem getting involved in this type of work? I think a very, probably the most promising direction, which is, uh, uh, something maybe I would like to, to explore a little bit is to, as I said, there is to go behind statical models. So everything I've shown, I've shown so far is, uh, is about static data. So the, the data don't change over time. There's no time inside the definition of predictive coding as it is, as I presented it here. However, you can, for example, generalize predictive coding to, to work with temporal data using generalized coordinates, as you mentioned earlier, uh, by, by presenting it as a, as a common common filter generative model. And, and that's where, for example, the, the causal inference direction could be very useful because at that model, uh, at that point, maybe you can be able to model Granger causality and, uh, and more complex and, and useful, uh, dynamical causal models, basically, because in general, the, the Ducalculus and the interventional and counterfactual, uh, branch of science is mostly developed on, uh, on small models. So it's, uh, like you don't do, uh, interventions on gigantic models in general. So if you, if you look at medical data, they, they use relatively small Bayesian networks and, uh, but of course, if you want to have a dynamical causal model that models, uh, a specific environment or a specific reality, you have a lot of neurons inside, you have a lot of latent variables, they change over time and an intervention at some, at some moment, uh, creates an effect in a different time step. So maybe in the next time step, in 10 different, different time steps later. And I think that would be very interesting to develop like a biologically plausible way of passing information, uh, that is also able to model Granger causality, basically. Hmm. Where do you see action in these models?

Where do I see action? I didn't think of that. I think I see actions in those models, maybe in the same way as I, as you see in other models, because creative coding is basically a model of perception. So, so an, an action is, you can see it as a consequence of what you are, uh, experiencing. So by changing the, the way you're, you're experiencing something, then you can compute, maybe, maybe you can simply perform a smarter action now that you have more information.

But, but yeah, I don't think action is very, is it, is it like, yeah, I don't see any explicit consequence of actions besides the fact that this can allow you to basically, maybe to, to simply draw better conclusions to then perform actions in the future.

I'll add onto that a few ways that people have, uh, talked about predictive coding and action. Uh, first off internal action or covert action is attention. So we can think about perception as an internal action. That that's one approach. Another approach pretty micro is the outputs of a given node. We can understand that node as a particular thing with its own sensory, cognitive, and action states. And so in that sense, the, the output of a node. Um, and then lastly, which we explored a little bit in live stream 43 on the theoretical review on predictive coding, we were reading all the way through, um, and it was all about perception, all about perception. And then it was like section 5.3. If you have expectations about action, then action is just another variable in this architecture. And that's really aligned with inactive inference where instead of having like a reward or utility function that we maximize, we select action based upon it being the likeliest course of action, the path of least action that's Bayesian mechanics. And so it's actually very natural to bring in an action variable and utilize it essentially as it, as if it were, uh, a prediction about something else exteroceptively in the world, because we're also expecting action.

No, yes, yes, exactly.

No, I like the way of defining actions a lot, actually. And, uh, and I still think it has been like, for example, there are not so many papers that apply this method. I think there are a couple from, uh, from Alexander or Obria does something similar, but in practice, like outside of the pure active inference, like applying predictive coding and actions to, to solve practical problems hasn't been, uh, explored a lot.

Cool. Well, thank you for this excellent presentation and discussion. Is there anything else that you want to, um, say or, or point people towards?

Uh, no, just a big thank you for inviting me. And, uh, it was really fun and I hope to come back at some point for, for some future works.

Cool. Anytime, anytime. Thank you, Tomaso.

Thank you, Daniel.

See you. Bye.

Bye.

Bye.

Bye.

Thank you.